GenMedia at Scale: Production-Grade Generative Media for Commerce

TL;DR

Generative AI models can produce stunning media — but getting them to work reliably at catalog scale is a different challenge entirely. This technical pattern describes the architecture and methodology for building **production-grade generative media pipelines** — the kind that produce 360-degree product spins, virtual try-on experiences, and contextualized product imagery across entire e-commerce catalogs with minimal human review.

The core insight: **the generation call is the easy part.** The hard part is everything around it — preparing inputs so models succeed on the first try, and automatically evaluating outputs so no human needs to review them.

The Problem

Retailers need rich product media — videos, contextual images, try-on experiences — across catalogs of thousands or millions of SKUs. Manual production doesn't scale. Generative AI should be the answer, but raw model outputs are unpredictable: a spinning video might reverse direction mid-spin, a try-on image might change the model's face, a product rendering might hallucinate extra buttons on a jacket.

At scale, raw model outputs require human review to catch failures — and when you're generating media across thousands of SKUs, that review becomes a bottleneck that defeats the purpose of automation. The business case demands fully automated pipelines that can detect, retry, and resolve quality issues without human intervention.

Technical Patterns

The architecture is built on three pillars that apply across all generative media use cases:

1. Input Optimization

> [Part 2: The Architecture Framework]

A toolkit of techniques — extraction, enhancement, classification, selection — composed differently for each use case. The goal: give the model the best possible starting point so it produces correct output more often.

2. Guided Generation

> [Part 2: The Architecture Framework]

Let reference images lead. Keep text descriptions minimal for generation — just enough to set the scene, not enough to conflict with visual references. The model should look at the images, not try to reconstruct details from text.

3. Automated Evaluation

> [Part 3: Deep Dive on Evaluation]

The hardest and most important piece. Every generated asset is automatically scored using a combination of deterministic deep learning models, LLM-as-judge evaluations, and ground truth comparisons. Assets that fail are retried or discarded — no human in the loop.

Document Structure

Part	Title	Focus
Part 1	Why GenMedia is Hard at Scale	Business case, core challenges, why "just call the API" fails
Part 2	The Architecture Framework	Three pillars, input optimization as a toolkit, the generate-evaluate loop
Part 3	Deep Dive: Evaluation	Evaluation methods, strategies, composite scoring, retry budgets
Part 4	Case Study: Product 360 Spinning	Generic product spinning pipeline with two-level validation

Part 5	Case Study: Footwear Spinning	Specialized footwear pipeline with semantic validation

---

Key Technologies

Component	Technology	Role
Video Generation	Veo 3	Reference-conditioned video synthesis
Image Generation	Nano Banana	Virtual try-on image synthesis
Vision Evaluation	Gemini	Glitch detection, garment accuracy, product consistency
Image Enhancement	Imagen 4 Upscale	Reference image upscaling
Face Similarity	**Face embedding model**	Deterministic face embedding comparison
Rotation Detection	**Optical Flow (Lucas-Kanade)**	Deterministic spin direction classification
Segmentation	Vertex AI Image Segmentation	Background removal and object extraction

---

Quick Impact

Metric	Impact
Conversion	Shoppers are 144% more likely to add-to-cart after engaging with video or contextualized imagery*
Returns	42% of returns happen because the product "looked different than expected"*
Cost	Media generation costs reduced to a fraction of manual production
Scale	Fully automated pipelines across entire catalogs — no human review required

**Source: Shopify 2025*

Part 1: Why GenMedia is Hard at Scale

The Business Case

E-commerce is a visual-first medium. The quality and richness of product media directly impacts two critical business metrics: **conversion** and **returns**.

Revenue impact. Shoppers who engage with immersive product media — 360-degree spins, virtual try-on, contextual lifestyle imagery — convert at dramatically higher rates. Across the industry, engagement with rich media correlates with a 144% increase in add-to-cart rates.* The logic is straightforward: customers who can see a product from every angle, or see how a jacket looks on a body similar to theirs, buy with more confidence.

Return reduction. Returns are one of the most expensive problems in e-commerce. 42% of all returns happen because the product "looked different than expected."* For apparel retailers operating on thin margins, this represents billions in lost revenue, shipping costs, and processing overhead. Better product visualization closes the gap between expectation and reality.

Operational efficiency. Traditional product media production — studio shoots, model bookings, post-production editing — costs anywhere from $50 to $500+ per SKU. For a retailer with 100,000 SKUs, generating 360-degree spins and try-on images through manual production is simply not feasible. The cost and logistics make it a subset-only activity: "We can only afford to shoot on-site for a fraction of the catalog."

Generative AI should solve this. Models like Veo, Gemini, and Imagen can produce photorealistic media from product photographs. But the gap between a single impressive demo and reliable production at catalog scale is vast.

**Source: Shopify 2025*

The Three Core Challenges

Calling a generative model API is easy. Getting it to produce consistently correct, high-fidelity output across thousands of diverse products — with zero human review — is where the real engineering lives.

Challenge 1: Fidelity

Generative models are creative by nature. That creativity becomes a liability when the goal is accurate product reproduction. Models hallucinate — they add, remove, or transform details that should be faithfully preserved.

The problem manifests differently per use case:

- **Product spinning:** A video of a phone might render the screen on both sides, or a backpack might grow an extra pocket that doesn't exist on the real product. Features appear and disappear as the product rotates.
- **Virtual try-on:** The model's face might change subtly — same person, but different enough to feel uncanny. Garments might be missing entirely, replaced by the original clothing.
- **Background transformation:** A product placed in a lifestyle scene might change proportions, or the model might "improve" the product by adding features that don't exist.

These aren't edge cases. With current-generation models, fidelity issues appear in a meaningful percentage of generations. At scale, that translates to thousands of unusable assets.

> **Key Insight:** Fidelity problems are fundamentally about the model filling in details from its training distribution rather than faithfully reproducing the reference. The architecture must minimize opportunities for the model to improvise.

Challenge 2: Brand Integrity and Product Consistency

Even when the model doesn't hallucinate outright, it can subtly drift from the product's actual appearance in ways that damage brand trust.

Retailers invest heavily in brand identity — specific colors, materials, hardware, logos, and design details that define their products. Generated media must preserve these details faithfully.

Where brand integrity breaks down:

- **Color and material drift:** A navy blue product that shifts toward black. A matte leather finish that becomes glossy. Subtle but brand-damaging differences that accumulate across a catalog.
- **Logos and branding:** Text that becomes illegible, mirrored, or repositioned. Brand-specific design elements that the model "smooths out" or replaces with generic alternatives.

- **Design detail simplification:** A distinctive crosshatch pattern on a heel counter replaced with something generic. Hardware (zippers, buckles, clasps) that loses its specific shape. The model defaults to "average" rather than preserving what makes the product unique.

Generative models are stochastic — they fill gaps from their training distribution, not from the product's specification sheet. Without explicit controls, every generation is an opportunity for brand-damaging drift.

Challenge 3: Reliability

At catalog scale, the pipeline must be autonomous. A retailer processing 10,000 SKUs cannot have a human reviewing each output for quality.

Reliability requires solving several sub-problems:

- **Automatic quality assessment:** Every generated asset must be evaluated without human intervention. This means building evaluation systems that can detect the fidelity and consistency problems described above.
- **Graceful failure handling:** When a generation fails quality checks, the system must decide whether to retry (and how many times), fall back to a different approach, or exclude the product. These decisions have cost implications — each retry burns API credits and processing time.
- **Bounded retry budgets:** More retries don't guarantee convergence — some inputs simply won't produce acceptable output regardless of how many times you regenerate. The architecture must cap retries, detect when a product won't converge, and decide whether to skip it or accept the best available result.

Why "Just Call the API" Doesn't Work

A naive approach to generative media looks like this:

```
Input Image → API Call → Output Media
```

This works for demos. It fails at scale for three reasons:

1. Input quality and metadata vary wildly. Product images from retailer catalogs come in every imaginable quality level: different resolutions, backgrounds, lighting conditions, angles, and compositions. Some products have 10 high-quality studio shots; others have a single low-resolution image on a cluttered background. Worse, images are often unlabeled — you don't know the product category, the viewpoint, whether it's cropped, worn by a person, or

showing an unusable angle (like the sole of a shoe instead of the side). A pipeline that assumes clean, labeled inputs will break on real-world catalog data.

2. No feedback loop. A single API call has no way to verify its own output. Did the spinning video actually rotate clockwise? Did the try-on image preserve the model's face? Did the product maintain its correct proportions? Without evaluation, every output is a guess.

3. No error recovery. Generative models are non-deterministic. The same input might produce a perfect result on one call and a flawed result on the next. Without retry logic informed by quality assessment, there's no mechanism to recover from bad generations.

The Production Architecture

The solution is not a single model or a single API call. It's an **architecture** — a composition of multiple models, traditional computer vision, and orchestration logic that turns unreliable generative calls into reliable production pipelines.

The architecture is organized around three pillars:

Pillar	Purpose	Analogy
Input Optimization	Prepare inputs so the model succeeds more often	Mise en place — prep before cooking
Guided Generation	Constrain the model to produce what we want	A recipe — specific instructions, not free improvisation
Automated Evaluation	Verify output quality and trigger retries	Quality control on the production line

These pillars are universal. They apply whether the use case is product spinning, virtual try-on, background transformation, or any future generative media pipeline. The specific techniques within each pillar vary by use case, but the architecture remains the same.

Part 2: The Architecture Framework

The Three Pillars

Every production-grade generative media pipeline is built on the same three-pillar framework. The pillars are sequential: optimize inputs, guide generation, evaluate outputs. But the system is a loop, not a line: failed evaluations trigger retries with the same optimized inputs, and in some cases evaluation results inform the next generation attempt.

Pillar 1: Input Optimization

Input optimization is not a fixed sequence of steps. It's a **toolkit** — a set of techniques that are composed differently depending on the use case and the quality of available inputs.

The Toolkit

Technique	What It Does	When It's Used
Extraction	Isolates the subject from its background	Removes noisy backgrounds that confuse generation models
Enhancement	Upscales resolution, cleans artifacts	When input images are low-resolution or have compression artifacts
Classification	Identifies properties of the input (viewpoint, category, pose)	When the pipeline needs to route inputs, validate coverage, or filter unusable images
Selection	Chooses the best subset of inputs from available options	When multiple input images are available and quality/coverage varies

> **Key Principle:** Input optimization is about reducing variance in what the model sees. The cleaner and more consistent the input, the less room the model has to improvise — and improvisation is the primary source of errors.

Pillar 2: Guided Generation

The generation call itself is where many teams focus their effort — prompt engineering, parameter tuning, model selection. In this framework, generation is deliberately kept **simple and constrained**. The complexity lives in the pillars on either side.

Reference Images Lead

For most generative media use cases, the reference images are the primary input. The model should reproduce what it sees in the images, not what it interprets from text.

> **Key Principle: Descriptions for generation should be minimal.** Let the images lead. A short, generic description ("A red ceramic mug") is sufficient to anchor the model. Detailed descriptions ("A red ceramic mug with a glossy finish, a curved handle on the right side, and a small chip near the base") risk conflicting with the visual reference — and when text conflicts with images, models often follow the text, introducing errors.

The Role of Descriptions: Generation vs. Evaluation

This is perhaps the most counterintuitive insight in the framework:

For generation: Descriptions should be *minimal*. Just enough context for the model to understand the scene, not enough to override the visual references.

For evaluation: Descriptions should be *rich and detailed*. A thorough description of the product — materials, hardware, logos, structural details — provides the vocabulary for evaluation models to assess fidelity. "Does the generated image show the distinctive crosshatch pattern on the heel counter?" can only be asked if the evaluation knows about the crosshatch pattern.

This asymmetry is deliberate. Generation models work best when visually guided. Evaluation models work best when given explicit criteria to check against.

Pillar 3: Automated Evaluation

Evaluation is the pillar that makes the framework production-grade. Without it, every output is a gamble. With it, the pipeline can achieve quality levels that approach or exceed human review.

The evaluation system is covered in depth in [Part 4: Deep Dive on Evaluation](). Here, we outline the architectural role it plays.

The Generate-Evaluate Loop

The core production loop is:

```
Generate → Evaluate → Accept or Retry
```

Each evaluation produces one of three outcomes:

Outcome	Action
Accept	Output passes all quality gates — deliver to production
Retry	Output fails a quality gate — regenerate within the retry budget
Discard	Output fails hard quality floors — cannot be used regardless of score

Retry Budgets

Retries don't guarantee convergence — some inputs simply won't produce acceptable output. The framework enforces **retry budgets** that cap attempts and define what happens when the budget is exhausted: either use the best available result (best-effort) or exclude the product entirely (strict). The choice is a business decision that trades quality against cost and coverage.

Evaluation Informs Generation

In advanced pipelines, evaluation doesn't just accept or reject — it **informs the next attempt**. If evaluation detects a specific deficiency in the output, it can trigger a targeted correction pass rather than a blind retry. This creates a multi-step self-correction loop:

```
Generate → Evaluate → Correct Specific Issue → Re-Evaluate → Pick Best Step
```

This pattern is more effective than retrying from scratch, because it builds on a partially-correct result and gives the model specific feedback about what went wrong.

Putting It Together

The three pillars compose into a general-purpose framework that can be instantiated for any generative media use case.

What changes across use cases is the specific composition of techniques within each pillar — which is exactly what the case studies in Part 4 and Part 5 demonstrate.

Part 3: Deep Dive — Evaluation

Why Evaluation is the Hardest Part

Input optimization and guided generation are engineering challenges with known techniques. Evaluation is fundamentally harder because it requires the system to **judge quality** — a task that traditionally requires human perception.

The evaluation system must answer questions like:

- Does this generated video contain visual artifacts or discontinuities?
- Does this generated image faithfully reproduce the reference product?
- Has the model preserved identity, structure, and brand-specific details?

These questions span a spectrum from purely objective (rotation direction is measurable) to deeply subjective (does this "look right"?). The evaluation framework must handle the full spectrum.

The stakes are high. A false positive — accepting a bad generation — means a defective asset reaches the customer-facing catalog. A false negative — rejecting a good generation — means wasted compute and potentially failing to produce media for a product. The evaluation system needs to be both sensitive enough to catch real problems and robust enough to avoid rejecting acceptable output.

Evaluation Method Types

The framework employs three categories of evaluation methods, each suited to different aspects of quality assessment.

1. Deterministic Deep Learning Models

What: Traditional deep learning and computer vision models that produce repeatable, objective measurements.

When to use: When the quality dimension can be quantified numerically — identity preservation, motion analysis, structural similarity.

Applicable methods:

Method	What It Measures	Output

Face embedding similarity	Whether a generated face matches the reference person	Cosine similarity (0–100%)
Optical flow analysis	Motion direction, consistency, and anomalies in generated video	Classification + confidence
Structural similarity (SSIM)	Pixel-level similarity between reference and generated images	Similarity score (0–1)

> **Key Advantage:** Deterministic models are fast, cheap (no API call), and reproducible. The same input always produces the same evaluation. They form the foundation of the evaluation stack and should be used as the first line of assessment before more expensive methods.

2. LLM-as-Judge with Gemini

What: Large vision-language models that analyze generated media and produce structured quality assessments.

When to use: When the quality dimension requires semantic understanding — detecting visual artifacts that break physical plausibility, assessing whether product details are accurately reproduced, identifying hallucinated features.

Applicable methods:

Method	What It Measures	Output
Visual artifact detection	Glitches, discontinuities, unnatural transformations in generated media	`{is_valid: bool, explanation: str}`
Product accuracy scoring	Whether reference product details are faithfully reproduced	Score on a defined scale (e.g., 0–100)
Multi-view product consistency	Whether a generated video maintains consistent product appearance across frames and matches the reference	`{is_valid: bool, explanation: str}`

Visual artifact detection sends generated media to Gemini and asks it to identify specific categories of problems: discontinuities, features appearing or disappearing, unnatural transformations. The prompt must be carefully calibrated to distinguish between actual defects and acceptable imperfections (minor wobbles, slight lighting variations, natural reflections).

Product accuracy scoring evaluates how faithfully a reference product is reproduced in the generated output. The scoring scale should be designed around **critical decision boundaries** — distinguishing between "completely wrong" (automatic discard), "present but imperfect" (usable with lower score), and "accurate reproduction" (high score).

Multi-view ground truth comparison is particularly powerful for consistency validation. Rather than checking individual outputs in isolation, it sends all reference images and matched generated outputs to Gemini in a single call, asking it to evaluate consistency across the entire set. This catches problems that single-view evaluation misses — like a product that looks correct from one angle but wrong from another.

> **Key Principle: Descriptions serve evaluation, not generation.** This is where rich, detailed product descriptions become valuable. While generation prompts should be minimal to avoid conflicts with reference images, evaluation prompts benefit from exhaustive descriptions. "Check whether the distinctive crosshatch pattern is present on the heel counter" requires knowing about the crosshatch pattern — and that knowledge comes from a detailed product description, not from the generation prompt.

Evaluation Strategies

The method types above are combined into **strategies** — patterns for how evaluation results drive pipeline behavior.

Strategy 1: Pass/Fail with Retry

Pattern: Generate → Evaluate → If fail, regenerate (up to N attempts)

Best for: Use cases where quality is binary — the output is either acceptable or it isn't.

Evaluations are **sequenced by cost:** cheap deterministic checks (local computation, no API call) run before expensive LLM evaluations. This saves money on obviously bad generations that can be caught without a Gemini call.

Strategy 2: Score-and-Rank with Parallel Variations

Pattern: Generate N variations in parallel → Evaluate all → Select the best

Best for: Use cases where quality is a spectrum, and the best result among multiple attempts is desired.

This strategy trades compute cost for quality — generating N variations costs Nx but dramatically increases the probability of producing at least one excellent result. It also fundamentally changes the latency profile: parallel generation achieves quality improvement in 1x wall-clock time instead of Nx sequential retries.

Strategy 3: Multi-Step Self-Correction

Pattern: Generate → Evaluate → Generate targeted correction → Re-evaluate → Pick best step

Best for: Use cases where specific, identifiable aspects of the generation can be corrected without starting from scratch.

Rather than discarding a partially-correct result and regenerating entirely, the pipeline detects the specific deficiency and triggers a correction pass. This is more efficient than blind retry because it builds on what was already correct and gives the model specific feedback about what went wrong.

Combining Strategies

These strategies are not mutually exclusive — they can be composed. For example, a pipeline might generate N variations in parallel (Strategy 2), apply a targeted correction pass to each variation (Strategy 3), and then score-and-rank the results. Or a pass/fail retry loop (Strategy 1) might include a self-correction step before deciding to discard and regenerate from scratch. The right combination depends on the use case, the cost profile, and which quality dimensions are most likely to fail.

Acceptance Thresholds and Composite Scoring

When multiple evaluation dimensions are assessed, they must be combined into a single actionable decision. The framework applies **acceptance thresholds** first, then **weighted composite scoring** on the survivors.

Acceptance Thresholds

Before computing any composite score, each evaluation dimension is checked against a hard floor. If any single dimension falls below its acceptance threshold, the output is discarded immediately — regardless of how well it scores on other dimensions. This prevents critically flawed outputs from being masked by high scores elsewhere.

Weighted Combination

Outputs that pass all acceptance thresholds are then scored. Multiple evaluation scores are combined with configurable weights that reflect business priorities:

```
final_score = dimension_1_score * weight_1 + dimension_2_score * weight_2 +
```

The weights encode what matters most for the specific use case. Adjusting weights shifts the quality tradeoff without changing the evaluation methods themselves.

Retry Budgets and Cost/Quality Tradeoff

Every retry costs money — API credits, compute time, and latency. The framework makes the cost/quality tradeoff explicit through **retry budgets**.

Cost-Aware Sequencing

Different evaluation methods have vastly different costs. The framework sequences evaluations from cheapest to most expensive:

Cost Tier	Examples	When to Use
Negligible	Embeddings, optical flow, SSIM	First — catch obvious failures for free
Low	LLM evaluation (Gemini)	Second — semantic checks on candidates that passed cheap checks
High	Regeneration (Veo, Gemini image generation)	Last resort — only when evaluation confirms the output is unacceptable

This sequencing ensures that expensive regeneration only happens when cheap checks can't resolve the quality question.

Budget Exhaustion

When the retry budget is exhausted, the pipeline must decide: use the best available result (best-effort) or exclude the product entirely (strict). This is a business decision that trades quality against catalog coverage. Both strategies are valid depending on the use case and quality requirements.

Evaluation Calibration

Evaluation models — especially LLM-as-judge — require careful calibration to distinguish between real problems and acceptable imperfections.

Strict vs. Lenient Criteria

Every evaluation prompt must define what constitutes a real defect versus an acceptable artifact:

Strict (Must Be Correct)	Lenient (Can Vary)
Product structure and features	Minor texture inconsistencies
Color accuracy	Slight lighting variations
Feature placement	Text legibility and spelling
Hallucinated or missing elements	Logo fine detail

The distinction is critical. Being too strict produces excessive false rejections (wasted compute). Being too lenient lets defective assets through. Calibration is an iterative process informed by analyzing which kinds of imperfections actually impact the end-user experience.

> **Key Principle:** Text and logos in generated images should be treated as "visual blobs." Correct position and approximate color is sufficient; exact legibility is not required. Generative models struggle with text rendering, and holding them to a legibility standard would reject otherwise excellent outputs.

Part 4: Case Study — Product 360 Spinning

The Use Case

Product 360-degree spinning videos are one of the highest-impact media types in e-commerce. They let shoppers view a product from every angle, dramatically increasing purchase confidence and reducing returns.

Traditionally, creating a 360 spin requires a motorized turntable, controlled lighting, and a camera capturing dozens of frames per rotation — followed by post-production stitching. This costs $100–500+ per product and limits coverage to high-value SKUs.

The generative approach: take a small number of static product photographs and generate a smooth, continuous 360-degree spinning video using a video generation model (Veo) with reference-image conditioning. No turntable, no studio, no post-production.

This section walks through how the three-pillar framework applies to generic product spinning (mugs, backpacks, electronics, watches, etc.). The specialized footwear pipeline is covered in Part 5.

Applying the Framework

Pillar 1: Input Optimization

The pipeline accepts multiple product images and applies the input optimization toolkit:

- **Extraction:** Each image is segmented to isolate the product and place it on a clean, uniform background. This prevents the generation model from reproducing or reacting to studio backgrounds, gradients, or clutter.
- **Enhancement:** Images are upscaled to provide the generation model with maximum visual detail. Higher-resolution references produce sharper, more faithful video output.

For generic products, classification and selection use pretrained models like Gemini to identify usable images, filter out unsuitable ones, and select the best references for generation.

Pillar 2: Guided Generation

The generation follows the framework's core principle: **reference images lead, descriptions are minimal.**

The model receives:

- Reference image canvases showing the product from different angles
- A short text prompt describing the action (e.g., "a continuous 360-degree orbit") and scene (e.g., "a white studio void")
- A brief subject description (e.g., "a red ceramic mug") generated by Gemini — just the product type and primary color, nothing more

The text prompt describes the *scene and motion*, never the product details. Product details come from the images. This minimizes the chance of text-image conflicts that lead to artifacts.

Example Veo prompt:

```
 [Subject]: A red ceramic mug standing still in a completely white studio voi
(Hex: #FFFFFF, RGB: 255, 255, 255).

[Action]: The camera performs one continuous, seamless, very fast 360-degree
around the stationary product. The camera movement is perfectly smooth and st
maintaining a constant distance and speed throughout the entire clip. The pro
does not move or rotate; only the camera moves.

[Scene]: A completely white studio void (Hex: #FFFFFF, RGB: 255, 255, 255).
The only visible element is the product, nothing else.
```

Notice how the prompt says nothing about the mug's handle, glaze, or any detail — it only describes the type, the camera motion, and the environment. The reference images carry everything else.

Pillar 3: Automated Evaluation

Every generated video passes through a **two-level validation** that applies the cost-aware sequencing principle from the evaluation framework.

Two-Level Validation

The key architectural pattern in this pipeline is layered evaluation: a cheap deterministic check first, then an expensive semantic check only if the first passes.

Level 1: Rotation Direction (Deterministic)

The first check uses **motion tracking** to determine whether the video shows the expected rotation direction. It tracks visual features across consecutive frames, measures how they move horizontally, and classifies the overall motion as clockwise, anticlockwise, or invalid. This is a pure computer vision approach — no API calls, no cost, sub-second execution.

> **Why not use an LLM for this?** Rotation direction is an objective, measurable property. A deterministic algorithm is faster, cheaper, and more reproducible. Beyond cost, LLMs are often not well suited for reasoning across frames about precise spatial properties like position, direction, or specific movements — they excel at semantic understanding, not pixel-level measurement.

Level 2: Visual Artifact Detection (Gemini)

Videos that pass the rotation check proceed to **semantic artifact detection** using Gemini as a vision evaluator.

Gemini evaluates the video for problems that optical flow can't catch:

Detect (INVALID)	Accept (VALID)
Hallucinated features (extra buttons, missing elements)	Minor wobbles or hesitations
Text/logos appearing mirrored or duplicated	Slight lighting variations
Product appearance changing mid-video	Natural reflections and shadows
Unnatural transformations or morphing	Minor surface imperfections

The prompt calibration — distinguishing real defects from acceptable artifacts — is critical. Too strict and the pipeline wastes compute rejecting usable output. Too lenient and defective videos reach the catalog.

The Retry Loop

If either validation level fails, the video is regenerated within a bounded retry budget. If the retry budget is exhausted, the pipeline can either use the best available result (best-effort) or skip the product entirely (strict), depending on quality requirements.

Conclusion

The generic spinning pipeline demonstrates the framework at its simplest: clean the inputs, keep the prompt minimal, and let layered evaluation — cheap deterministic checks first, semantic judgment second — turn a non-deterministic generation model into a reliable production system. When a product category demands deeper control, the same architecture extends naturally — as the footwear case study shows next.

Part 5: Case Study — Footwear Spinning

Why Shoes Are Harder

Footwear is one of the most challenging product categories for AI-generated spinning videos. Where a generic product (a mug, a backpack) has few well-defined viewpoints, a shoe has a highly structured geometry with **distinct, named views** — right side, left side, front, back, sole, top — that customers know and expect to see.

This creates problems that don't exist for generic products:

1. Pair ambiguity. Product photos frequently show two shoes together. The generation model needs a single shoe to spin, which means the pipeline must detect pairs and split them into individual shoe images.

2. Viewpoint coverage requirements. A spinning video must show the shoe from all angles. If the input images only cover the right side and front, the model must hallucinate what the back and left side look like — and it will often get it wrong (mirroring logos, inventing stitching patterns). The pipeline needs to verify that input images cover all cardinal viewpoints before attempting generation.

3. Rotation verifiability. Because shoes have named viewpoints, the pipeline can do something impossible with generic products: verify that the generated video actually passes through the correct sequence of views. A clockwise rotation should show: right → front → left → back → right. This enables **semantic spin validation** — a much stronger quality check than motion tracking alone.

How the Framework Applies

The footwear pipeline demonstrates the full power of the three-pillar framework. Every technique in the toolkit is used, and the evaluation strategy combines multiple methods.

Input Optimization: Classification-Driven

Unlike generic products, footwear requires **deep classification** before any processing begins.

Viewpoint classification. Catalog images arrive unlabeled and vary widely — you might get clean side views, but also intermediate angles (front_right, back_left), cropped close-ups, zoomed-in details, images of someone wearing the shoes, or sole views. Every input image is classified by a fine-tuned model into categories covering cardinal views (right, left, front,

back), compound views (front_right, back_left, etc.), and special categories (sole, multiple shoes, invalid). Invalid images — cropped, zoomed, worn by a person, showing the sole — are discarded before any expensive processing.

Pair splitting. Images containing two shoes are automatically detected through classification and split into individual shoe images using segmentation. The resulting images are re-classified to determine their viewpoints.

Feasibility check. After classification, the pipeline verifies that the input set covers all four cardinal views (front, back, left, right) — compound views count toward their components (e.g., front_right covers both front and right). If coverage is insufficient, the product is excluded before any expensive processing occurs — a direct application of the fail-fast principle.

Selection and ordering. From the classified images, the pipeline selects the best images and orders them to match the clockwise rotation path. When multiple images cover the same viewpoint, the most complete one is selected. This gives the generation model the strongest possible visual references in the right sequence.

Guided Generation

Same principle as generic spinning: reference images lead, descriptions are minimal. The prompt structure is identical — subject, action, scene — with only the subject description adapted for footwear.

Example Veo prompt:

```
 [Subject]: A blue running shoe standing still in a completely white studio v
(Hex: #FFFFFF, RGB: 255, 255, 255).

[Action]: The camera performs one continuous, seamless, very fast 360-degree
around the stationary product. The camera movement is perfectly smooth and st
maintaining a constant distance and speed throughout the entire clip. The pro
does not move or rotate; only the camera moves.

[Scene]: A completely white studio void (Hex: #FFFFFF, RGB: 255, 255, 255).
The only visible element is the product, nothing else.
```

The prompt says nothing about the shoe's materials, stitching, or branding — all of that comes from the reference images.

Automated Evaluation: Multi-Layer

This is where the footwear pipeline diverges most from the generic version. Instead of motion tracking + artifact detection, it uses **three layers of evaluation**, each catching a different class of problem.

Semantic Spin Validation

This is the **core differentiator** of the footwear pipeline. Instead of measuring motion direction, it validates the spin by **classifying every sampled frame** and checking that the sequence follows a valid rotation path.

The Rotation Graph

The valid clockwise rotation is defined as a directed graph:

```
right → front_right → front → front_left → left → back_left → back → back_ri
```

How It Works

1. **Frame sampling:** Frames are sampled from the video at a reduced rate
2. **Frame classification:** Each sampled frame is classified using the shoe classifier in "validation" mode — a simplified set of categories excluding views that shouldn't appear in a proper spinning video (sole, multiple, etc.)
3. **Coverage check:** All 8 positions must appear in the classified frames. Missing positions indicate an incomplete rotation.
4. **Path validation:** The sequence of classifications is checked against the rotation graph, with tolerance for noise (individual misclassifications, oscillations at transition boundaries, skipped intermediate views)

Direction Detection

If the clockwise path check fails, the **reversed sequence** is checked. If the video is valid anticlockwise, the frames are reversed to produce clockwise output — the video is saved, not discarded.

Completeness Check

The pipeline also verifies that the video represents a **full 360-degree rotation**. If the video went past 360°, it's trimmed to exactly one rotation. If it's incomplete, it's flagged as invalid.

Product Consistency Validation

After spin validation passes, a second evaluation checks whether the **generated product matches the reference images** — detecting cases where the model hallucinated details, changed colors, or altered the shoe's structure.

The pipeline uses the **multi-view ground truth comparison** pattern from the evaluation framework: generated frames are matched to reference images by their classified viewpoint, and Gemini evaluates consistency across all views in a single call.

The evaluation criteria distinguish between strict and lenient checks:

Strict (Must Match)	Lenient (Can Vary)
Feature placement (buckles, straps, eyelets)	Text legibility and spelling
Structural integrity (sole shape, heel height)	Logo fine detail
Color consistency across views	Minor angle differences
Hallucinated or missing features	Generative artifacts (slight texture variation)

> **Key Principle:** Text and logos are treated as "visual blobs." Correct position and approximate color is sufficient — exact text rendering is not required.

If product consistency fails, the entire video generation is retried within the retry budget.

Frame Post-Processing

Once a video passes all validation layers, the frames are post-processed for final output:

- **Reordering:** The sequence is adjusted to start from a consistent viewpoint, matched to the closest reference image
- **Cropping and resizing:** Product bounds are detected across all frames and a uniform crop is applied, producing consistent framing throughout the rotation

The final output includes both the video and a set of sampled frames suitable for interactive 360 viewers on product pages.

Conclusion

The footwear pipeline illustrates a broader point about the framework: the same three pillars — input optimization, guided generation, automated evaluation — scale to arbitrarily complex product categories by composing more techniques from the toolkit. Classification becomes deeper, evaluation becomes multi-layered, and domain knowledge (like the rotation graph) unlocks validation strategies that generic approaches can't achieve. The architecture doesn't change; the techniques within it do.